

Anomaly Detection based on Isolation Forest and Local Outlier Factor

Abubakar Mastour Adam Fadul (abubakar@aims.ac.za)

African Institute for Mathematical Sciences (AIMS)

Supervised by: Prof. Bruce Bassett
African Institute for Mathematical Sciences(AIMS), South Africa
University of Cape Town, South Africa

25 May 2023

Submitted in partial fulfillment of a structured masters degree at AIMS South Africa



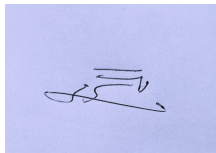
Abstract

Anomaly detection plays an essential role in detecting unusual patterns or outliers in data providing valuable insights and enabling proactive decision-making. With the increasing availability of large and complex datasets, the need for effective anomaly detection techniques becomes even more critical across various industries and research applications. In this research we compare two machine learning algorithms that have been used for anomaly detection: Isolation Forest (IForest) and Local Outlier Factor (LOF). We evaluate their performance using the Matthew Correlation Coefficient (MCC), Area Under the Curve, and execution time. Additionally, we discuss the advantages and disadvantages of each algorithm. To evaluate the performance of the two models we used seven different datasets from the University of California Irvine machine learning repository. The results showed that IForest outperforms LOF in terms of area under the curve and execution time and this is because IForest is based on the concept of random forests and uses an ensemble of decision trees to isolate outliers and LOF uses the density-based approach and calculates the local density of each data point, comparing it to the density of its neighbours to identify outliers. However, in terms of MCC, LOF outperforms IForest in some datasets, while IForest outperforms LOF in others, and in some datasets, they perform equally.

Keywords: Anomaly detection, isolation forest, outlier detection, local outlier factor, k-nearest neighbour, machine learning.

Declaration

I, the undersigned, hereby declare that the work contained in this research project is my original work, and that any work done by others or by myself previously has been acknowledged and referenced accordingly.



Abubakar Mastour Adam Fadul, 25 May 2023

Contents

Abstract	i
1 Introduction	1
1.1 Machine Learning Algorithm	1
1.2 Types of Machine Learning	1
1.3 Anomaly Detection Algorithms	2
1.4 Study Objectives	4
1.5 Research Structure	4
2 Literature Review	5
2.1 Isolation Forest(IForest)	5
2.2 Local Outlier Factor (LOF)	7
3 Methodology	9
3.1 IForest Algorithm	9
3.2 LOF Algorithm	13
3.3 Evaluation Metrics	16
4 Datasets	19
4.1 Satellite Dataset	19
4.2 Shuttle Dataset	19
4.3 Http(KDDCUP99) Dataset	20
4.4 Forest Cover Dataset	20
4.5 Ionosphere Dataset	21
4.6 Breast Cancer Wisconsin Dataset	21
4.7 Annthyroid Dataset	21
4.8 Splitting Datasets	22
4.9 Evaluation Stage	23
5 Results and Discussions	24
5.1 Results	24
5.2 The Strengths and Weaknesses of the Two Algorithms	25
5.3 Discussion	27
6 Conclusion and future work	28
6.1 Conclusion	28
6.2 Future Work	28
References	32

1. Introduction

1.1 Machine Learning Algorithm

Machine Learning (ML) is part of Artificial intelligence which focuses on developing algorithms and statistical models that help the computer learn from the data and make predictions, identify patterns, or classify data into different types [Heaton \(2018\)](#). Recently, ML has become very important in different domains such as:

- **Systems recommendation:** used to recommend services, content, or products to users based on their previous behaviour preferences.
- **Natural language processing:** used to analyse and understand the human language and this enables computers to perform tasks such as language translation and speech recognition.
- **Video and image analysis and classification:** used to identify and classify objects and this can help with object detection and image recognition.
- **Autonomous vehicles:** used to analyse data from sensors such as radar and camera enabling safety and decision making and this improves transportation efficiency and reduces accidents.
- **Predictive modelling in finance:** used to analyse financial data such as stock prices and economic indicators to make predictions and to optimise investment strategies.

All of these applications benefit from ML's ability to automate complex tasks and improve the accuracy and efficiency of various applications as well as analyse and identify patterns from large amounts of data in a short time, tasks which are challenging for humans to perform [Jordan and Mitchell \(2015\)](#).

1.2 Types of Machine Learning

Machine Learning can be classified into three different types: supervised, unsupervised, and reinforcement learning [Alpaydin \(2020\)](#).

1.2.1 Supervised Learning

The algorithm is supervised i.e. it is trained on a dataset where the correct output is known in advance for each input, and the main goal of the algorithm is to accurately predict the output using the mapping function for unseen input data [Alpaydin \(2020\)](#). Supervised learning can be subdivided into classification, where the output variable is a categorical variable, and regression where the output is a continuous variable and the prediction is numerical value [Hastie et al. \(2009\)](#).

1.2.2 Unsupervised Learning

The algorithm is unsupervised i.e. it looks for relations and patterns in unlabeled data without any prior knowledge or human guidance. The algorithm is not given any target variable and it has to learn from the inherent structure and relationships in the data.

Unsupervised learning can be subdivided into clustering, in which the data points are assigned to groups based on their characteristics, behaviour, and similarities; dimensionality reduction, where the input variables in the dataset are reduced because having more input features makes predictive modelling more challenging; and anomaly detection (outlier detection), which searches for significantly different items in the dataset and classifies them as anomalies.

The advantage of unsupervised learning is its ability to work with high-dimensionality datasets and achieve good performance. It can also identify hidden patterns and relationships in the dataset [Hastie et al. \(2009\)](#).

1.2.3 Reinforcement Learning(RL)

The algorithm develops a model that returns an appropriate solution for a problem by making a decision by itself. In RL, the agent interacts with the environment by taking action and receiving feedback in the form of rewards or penalties. This can help the agent to update its decision-making strategies and learn from its mistakes [Hastie et al. \(2009\)](#). RL is inspired by the way that humans and animals learn through trial and error and has many applications including robotics, gaming, recommendation systems, and autonomous vehicles. In robotics, RL can be used to train the agent to carry out complex tasks and learn from experience and feedback for future decisions. In gaming, RL can be used to develop intelligent agents that can play games at a human level and also can develop strategies for games. In recommendation systems, RL can be used to learn users' preferences and recommend products or services based on their previous activities. In autonomous vehicles, RL can be used to train vehicles to navigate complex environments and traffics, making safe and efficient decisions that reduce accidents. RL has several advantages: it can work with complex and dynamic environments, learn from experience, adapt to changing environments, improve decisions over time, and generalise to new situations among others [Sutton and Barto \(2018\)](#).

1.3 Anomaly Detection Algorithms

Anomaly detection algorithms are used to identify rare or unusual events in the data that deviate significantly from normal behaviour. There are various types of anomaly detection algorithms, including Isolation Forest (IForest), Local outlier Factor (LOF), One-Class Support Vector Machine (OCSVM), and K-Nearest neighbour (K-NN) [Chandola et al. \(2009\)](#). We only consider the first two algorithms in this essay IForest and LOF and this is because LOF is effective in detecting local outlier compare to other algorithms and IForest fast in detecting anomalies and this is because the algorithm isolate the anomalies rather than normal instances. The difference between anomaly detection and standard classification methods is that standard classification is to assign input data points to predefined classes or categories based on their features. It aims to learn a decision boundary that separates different classes. On the other hand, anomaly detection focuses on identifying unusual or abnormal data points that deviate significantly from the norm or expected behaviour. In terms of output; classification models assign a class label to each instance, indicating which category it belongs to based on the learned decision boundaries, while anomaly detection methods typically output a binary result, indicating whether an instance is normal or anomalous [Chandola et al. \(2009\)](#).

1.3.1 Isolation Forest(IForest) Algorithm:

IForest is an unsupervised learning technique that is built on decision trees where the algorithm detects anomalies by partitioning the data recursively using the random split (we will discuss this later on). It was proposed as an alternative to traditional anomaly detection methods by Fei Tony Liu and Zhi-Hua

Zhou in 2008. IForest is based on the intuition that the anomalies are easier to isolate than the normal instances which require low memory and this makes IForest fast and reliable. The algorithm builds an ensemble of randomized decision trees to isolate anomalies more efficiently than traditional methods for anomaly detection. IForest has been applied in different domains, such as fraud detection, intrusion detection, recommendation system, and anomaly detection.

IForest has many advantages over other algorithms, such as its ability to achieve good performance when dealing with high-dimensional data, reduce overfitting, and handle both continuous and categorical datasets [Liu et al. \(2008\)](#).

1.3.2 Local Outlier Factor(LOF) Algorithm:

LOF is an unsupervised learning technique for identifying anomalies in the datasets based on the density of the neighbourhood i.e. it compares the density of certain points' neighbours to identify outliers. An outlier in this algorithm is a data point that has a lower density compared to its neighbouring points. LOF is a density-based technique that detects anomalies based on their local density deviation from their neighbours, making it more effective than other algorithms when working with high-dimensional datasets. This algorithm has proven its ability to detect local outliers, which are data points that resemble normal instances. As shown in Figure(1.1), the points (a, h, i, and j) are considered local outliers because they have low densities compared to their neighbours and they are near to the clusters E, F, and G. Additionally, it can detect global outliers: as shown in Figure(1.1), the points (b, c, and d) are considered global outliers because they have low densities compared to their neighbours and they are far away from the clusters E, F, and G. The LOF algorithm has been successfully applied in various domains such as video and image processing and fraud detection [Breunig et al. \(2000\)](#).

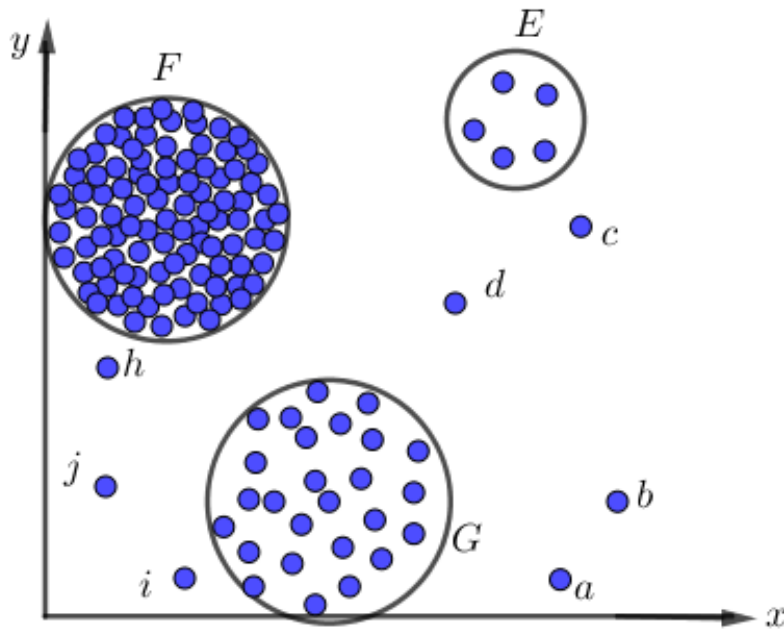


Figure 1.1: Two-dimensional data demonstrate local and global outliers.

1.4 Study Objectives

The objective of this essay is to closely examine two machine learning algorithms, Isolation Forest and Local Outlier Factor, and compare how the two models perform on different datasets and evaluate their performance using the Matthew Correlation Coefficient (MCC), the area under the curve (AUC), and compare their execution time, as well as introduces the strengths and weaknesses of the two algorithms.

1.5 Research Structure

This essay is structured as follows: Chapter 2 provides a literature review on the two algorithms and how some limitations are solved over time. Chapter 3 describes the methodology of the two algorithms, including how they work and the mathematical tools employed to obtain results, and how the code is implemented to obtain the results. Chapter 4 introduces the datasets used to evaluate the performance of the two algorithms. Chapter 5 discusses the results obtained from the algorithms and the advantages and disadvantages of the two algorithms as well. Chapter 6 concludes the essay and outlines future work.

2. Literature Review

Anomaly detection is an important task in machine learning and data mining applications. Anomalies are data points that differ significantly from the rest of the data [Chandola et al. \(2009\)](#). Figure(1.1) shows how anomalies appear in a two-dimensional dataset.

Anomaly detection is a very important domain in machine learning and has many applications in the real world as mentioned earlier. There are various algorithms that have been used to detect anomalies and these algorithms differ from each other, each with its own advantages and disadvantages. In this essay, we focus on two specific algorithms: IForest and LOF.

David Wolpert and William Macready in their article (“No Free Lunch Theorems for Optimization”) state that there is no perfect algorithm in machine learning that outperforms others in all types of problems. The performance of an algorithm depends on the nature of the problem and if the algorithm performs perfectly in one problem it may work poorly with another problem regardless of where the algorithm is adaptive or randomized [Wolpert and Macready \(1997\)](#). Therefore, neither IForest nor LOF is better than the other; their performance depends on the nature of the problem and the characteristic of the dataset.

2.1 Isolation Forest(IForest)

According to [Liu et al. \(2008\)](#), IForest is an unsupervised machine learning algorithm proposed by Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou in 2008 in their article “Isolation Forest”. This algorithm demonstrates high performance in detecting anomalies and is based on the concept of using random forests for this purpose. It works by constructing an ensemble of isolation trees to isolate anomalies. The performance of the IForest algorithm has been evaluated using various real-world datasets and compared with other anomaly detection algorithms, including ORCA, Local Outlier Factor (LOF), and Random Forest(RF). The results of this comparison are shown in table 2.1.

Datasets	AUC				Time (seconds)					
	iForest	ORCA	LOF	RF	iForest			ORCA	LOF	RF
					Train	Eval.	Total			
Http (KDDCUP99)	1.00	0.36	NA	NA	0.25	15.33	15.58	9487.47	NA	NA
ForestCover	0.88	0.83	NA	NA	0.76	15.57	16.33	6995.17	NA	NA
Mulcross	0.97	0.33	NA	NA	0.26	12.26	12.52	2512.20	NA	NA
Smtpt (KDDCUP99)	0.88	0.80	NA	NA	0.14	2.58	2.72	267.45	NA	NA
Shuttle	1.00	0.60	0.55	NA	0.30	2.83	3.13	156.66	7489.74	NA
Mammography	0.86	0.77	0.67	NA	0.16	0.50	0.66	4.49	14647.00	NA
Anthyroid	0.82	0.68	0.72	NA	0.15	0.36	0.51	2.32	72.02	NA
Satellite	0.71	0.65	0.52	NA	0.46	1.17	1.63	8.51	217.39	NA
Pima	0.67	0.71	0.49	0.65	0.17	0.11	0.28	0.06	1.14	4.98
Breastw	0.99	0.98	0.37	0.97	0.17	0.11	0.28	0.04	1.77	3.10
Arrhythmia	0.80	0.78	0.73	0.60	2.12	0.86	2.98	0.49	6.35	2.32
Ionosphere	0.85	0.92	0.89	0.85	0.33	0.15	0.48	0.04	0.64	0.83

Table 2.1: IForest performs favorably to ORCA, especially for large data sets where $n > 1000$. AUC reported with boldfaced are the best performance. iForest is significantly faster than ORCA for large data sets where $n > 1000$. We do not have the full results for LOF and RF because (1) LOF has a high computation complexity and is unable to complete some very high volume data sets in reasonable time; (2) RF has a huge memory requirement, which requires system memory of $(2n)^2$ to produce proximity matrix in unsupervised learning settings. [Liu et al. \(2008\)](#).

A study done by [Laskar et al. \(2021\)](#) proposed a new method for detecting anomalies by expanding the IForest algorithm and combining it with K-Means clustering. The aim of this study is to extend the IForest algorithm to detect anomalies in large-scale datasets by using K-Means clustering. The K-means algorithm is an unsupervised machine learning algorithm used for clustering data points into distinct groups based on their similarity. The objective of K-means is to partition a given dataset into K clusters, where each data point belongs to the cluster with the nearest mean value. The proposed algorithm works by first applying K-Means to the dataset to subdivide the dataset into small clusters and assign each point to a relevant cluster and then applying IForest for each cluster individually to identify anomalies, instead of applying it to the whole dataset. Then they evaluated the performance of the algorithm by testing it on two different datasets and comparing the performance with that of the original IForest algorithm and several other anomaly detection methods. This algorithm demonstrates good performance in terms of accuracy and computational efficiency compared to the original IForest algorithm and other anomaly detection methods.

A study done by [Tao et al. \(2018\)](#) proposed a parallel algorithm for IForest, which takes advantage of multi-core processors to increase the speed of the detection process by dividing the tasks between cores. This parallel algorithm works by partitioning the data into multiple subsets and processes, each subset uses a separate core to calculate the anomaly score, and anomaly scores for subsets are then combined to obtain the final anomaly score for the entire dataset. This algorithm was applied and tested using two different datasets. After comparing the result from the two datasets the authors found that the parallel algorithm was faster and more accurate in detecting anomalies. Additionally, they demonstrated how the parallel implementation of the IForest algorithm could scale effectively with growing data volumes making it appropriate for use in extensive anomaly detection applications.

A study done by [Marteau et al. \(2017\)](#) proposed a modified version of the IForest algorithm, which they called the “hybrid isolation forest”. This algorithm uses both categorical and continuous features in constructing isolation trees, the algorithm first partitions the data based on categorical features and then partitions each resulting subset based on continuous features. This approach allows the algorithm to handle mixed-type data more effectively. The performance of the hybrid isolation forest algorithm was evaluated using the ISCX dataset and compared with other anomaly detection algorithms. The results demonstrate that the hybrid isolation forest algorithm performed better than the other algorithms.

A study done by [Karczmarek et al. \(2020\)](#) proposed a modified version of the IForest technique called “K-Means-based Isolation Forest” (KMIF). The proposed algorithm is a combination of the K-Means and IForest algorithms. Because the traditional IForests may not be effective in detecting anomalies in data with complex structures, these two algorithms combined together to improve the performance of anomaly detection. The data points are initially clustered by K-means clustering according to how similar they are. The IForest algorithm then employed the cluster label that was assigned to each data point as an extra feature. This enables the IForest to take into account each data point’s position within a particular cluster in addition to how close it is to other points in a specific cluster. The cluster centres are used as the initial splitting points for the trees in each Isolation Forest and then recursively partition the data points in each cluster into smaller subsets using the random trees. The performance of KMIF is evaluated on several datasets, and it is compared with traditional IForests and other anomaly detection methods. KMIF demonstrates better performance than various other algorithms in terms of anomaly detection accuracy and processing efficiency, especially in datasets with high dimensionality and complicated structures. The KMIF method in comparison to the traditional isolation forest method, provides a number of advantages such as the K-means clustering step which allows the method to handle datasets with many clusters and where anomalies are found close to cluster boundaries. It also reduces

the computational complexity of the isolation forest algorithm by reducing the number of trees required to achieve a given level of accuracy.

A study done by [Lesouple et al. \(2021\)](#) proposed a new algorithm for anomaly detection called the Generalised Isolation Forest (GIF) and this algorithm is based on the original IForest algorithm. The idea behind the GIF algorithm is to use a set of splitting functions rather than just randomly selecting a feature and split value as in the original IForest algorithm. And the splitting functions are selected from a predefined set of functions and are chosen based on their ability to effectively partition the data. In addition to using a set of splitting functions, The GIF algorithm uses a more advanced approach to measure the distance between a point and the splitting functions, and this is accomplished by considering the magnitude and direction of the distance vector between the point and the splitting function. The GIF algorithm is more resilient to noise and outliers than the original IForest algorithm and can handle a wider range of data distributions. The GIF algorithm improved the performance of the original IForest algorithm by adding a new way of selecting the feature and split point at each step as well as handling different kinds of data and achieving high accuracy in anomaly detection.

A study done by [Liu et al. \(2012\)](#) proposed an approach using IForest called “Isolation-based Anomaly Detection” (IAD), this algorithm adds several improvements and modifications to the original IForest algorithm such as instead of choosing a random feature and splitting point, IAD chooses a small set of features to split on and then uses a weighted average of the split points of those features and this makes IAD handle datasets with high-dimensional features and outliers that are dispersed across multiple dimensions more effectively. In the IAD algorithm, each tree is constructed using a random subset of the dataset, as a consequence the algorithm decreases the overfitting problem, and has a lower computational complexity.

2.2 Local Outlier Factor (LOF)

The Local Outlier Factor (LOF) is an unsupervised machine learning algorithm proposed by Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander in 2000. It is used to identify outliers within a dataset by calculating the local density of a data point relative to its neighbours and a data point that is isolated from the rest of the data in its local neighbourhood is considered an outlier. When a data point has low neighbour density and is isolated, it is more likely to be an outlier. Conversely, When a data point with high neighbour density and is part of a cluster, it is likely to be a normal instance. This algorithm shows good performance particularly when dealing with high-dimensional data where traditional outlier detection algorithms may not perform well [Breunig et al. \(2000\)](#).

A study done by [Cheng et al. \(2019\)](#) discussed an approach to detecting outliers using two popular anomaly detection algorithms, IForest and LOF in their article “Outlier Detection using Isolation Forest and Local Outlier Factor”. They apply several datasets to the two algorithms and then compared the results obtained from the LOF algorithm with the results obtained from the IForest algorithm. They found that LOF achieved better performance than IForest in all metrics except for the execution time where IForest was faster, and to handle this problem they proposed to use optimized implementations or parallelization. This approach showed that LOF is a useful approach for detecting outliers, particularly in datasets with a high-dimensionality and complicated structures.

A study done by [Alghushairy et al. \(2020\)](#) discussed “A Review of Local Outlier Factor Algorithms for Outlier Detection in Big Data Streams”. They reviewed several LOF algorithms for outlier detection in big data streams. The purpose of this study was to compare the performance of several LOF algorithms on huge data streams with other anomaly detection algorithms, and the results showed that LOF algorithms

achieved good performance in detecting outliers in big data streams, with high accuracy compared to other anomaly detection algorithms, additionally, they mentioned that each algorithm has advantages and disadvantages in terms of computational complexity, scalability, accuracy, and parameter selection and some of the algorithms are based on density estimation, While other algorithms use clustering and distance-based techniques. The aim of these techniques is to precisely and effectively detect and rank the outliers in a dataset.

A study done by [Xu et al. \(2022\)](#) proposed a novel algorithm for outlier detection called “Outlier detection algorithm based on k-nearest neighbours-local outlier factor”. This algorithm combines the k-nearest neighbours (K-NN) algorithm with the LOF algorithm and this can be done by identifying k-nearest neighbours for each data point and then calculating the reachability distance and using these reachability distances to compute the local outlier score for each data point. To evaluate the performance of this model they conducted experiments on several datasets and compared this algorithm with several other anomaly detection algorithms, such as the original LOF algorithm, k-means clustering, and IForest. The result obtained showed that this algorithm outperforms the other anomaly detection algorithms in terms of detecting anomalies, efficiency, and accuracy.

A study done by [Wang and Chen \(2021\)](#) proposed a new version of the LOF algorithm called “Robust feature matching using guided local outlier factor” to enhance the efficiency of the feature matching mission. In feature matching the objective is to identify correspondences between several viewpoints of an object or a scene and this is a very difficult process for the original LOF to do because of the noise, outliers, and occlusions. They improve the LOF algorithm by adding a guidance mechanism that considers the information provided by a reference image. This guidance mechanism relies on the idea that features in the two images that are similar should have almost the same LOF scores specifically for each feature in one image they calculate the LOF score of its potential match in the second image. The LOF computation in the feature space is then guided by these scores, thereby biasing the algorithm to assign lower scores to potential matches that have different LOF scores. This algorithm was evaluated on several benchmark datasets for feature matching and compared the result with other anomaly detection algorithms, this algorithm outperformed the other anomaly detection algorithms in terms of accuracy and robustness to outliers.

A study done by [Vijayakumar et al. \(2020\)](#) proposed using IForest and LOF in their article “Isolation Forest and Local Outlier Factor for Credit Card Fraud Detection System” for detecting fraud in credit cards. They applied these two algorithms to a dataset of credit card transactions in order to detect fraudulent transactions and evaluate the performance of these algorithms. They compared the results from the two algorithms in terms of precision, recall, accuracy, and F1-score. They found that both algorithms are effective in detecting credit card fraud, with the IForest algorithm outperforming LOF in terms of accuracy and F1-score.

A study done by [ELHaddad et al. \(2023\)](#) proposed a new approach in their article “Comparison of Enhanced Isolation Forest and Enhanced Local Outlier Factor in Anomalous Power Consumption Labelling”, comparing the performance of the two well-known anomaly detection algorithms, IForest and LOF. To improve the performance they added a new feature to these two algorithms called “Enhanced Isolation Forest (E-IF) and Enhanced Local Outlier Factor (E-LOF)”. To evaluate the performance of the proposed algorithms (E-LOF and E-IF) they used a power consumption dataset, and they found that both algorithms performed well in detecting anomalies, with E-IF achieving slightly better results than E-LOF.

3. Methodology

3.1 IForest Algorithm

According to Liu et al. (2008) anomalies are “fewer and different” (this means the amount of anomaly in a dataset is always very small and they did not resemble the rest of the data points) from the rest of the dataset and can be easily isolated (near the root of the tree, as the data point *a* in figure 3.2), while normal points are isolated at the end of the tree. The algorithm classifies the point as an anomaly if the point has a short path length of the isolation tree.

To use this algorithm for anomaly detection there are two main steps:

The first step is to train the model; which builds an isolation tree (explained in section 3.1.1) by sub-sampling the dataset, and the second step is to test the model by passing the test dataset into the isolation tree to calculate the anomaly score for each point.

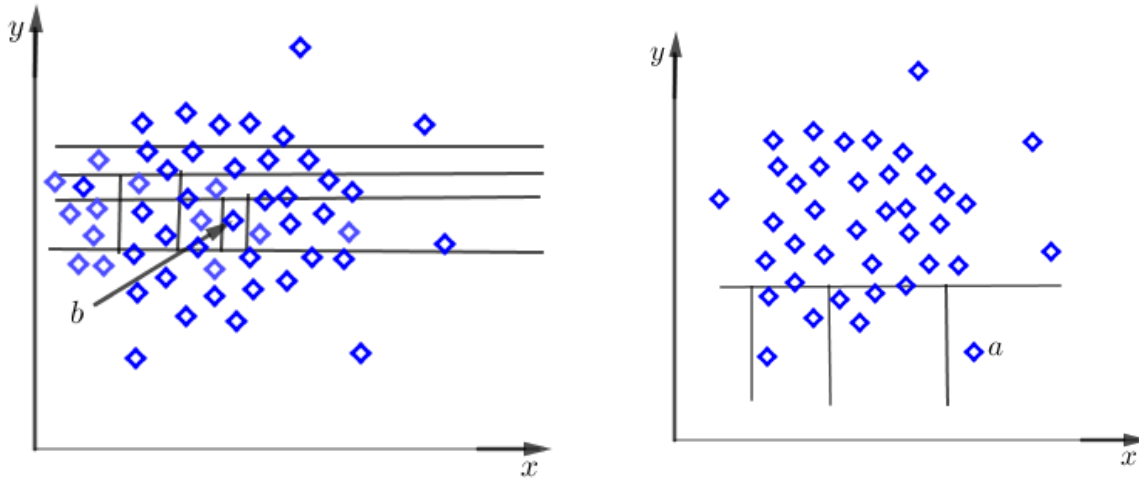


Figure 3.1: The figure on the right shows how an anomaly is isolated, and the figure on the left shows how a normal point is isolated.

3.1.1 Isolation Tree

The isolation tree algorithm works by randomly selecting a feature from the dataset and a value within the range of that feature then using that selected value and feature will split the data into two subsets, this will create two child nodes for the current node, one for each subset of data. The subset smaller than the split value will go to the left node and greater than the split value will go to the right as shown in figure(3.2) and this is repeated recursively on the subsets of data created by the split until either all the data points in a subset belong to the same class (i.e. normal point isolated in a separate node and anomaly in a separate node) or the maximum tree depth is reached. After the isolation trees are built, the IForest algorithm calculates the anomaly score for each data point by computing the average depth of the tree in which the data point is isolated then the algorithm will classify a data point as an anomaly if it has a high average depth and as a normal if it has a low average depth (the average depth will be obtained by calculating the path length of a data point over all trees and average them), i.e. data points with shorter path length are classified as anomalous and data points with long path length are

classified as normal points.

As shown in figure(3.2) the data point a was easily isolated in each isolation tree and has a shorter path length, while the data point b was isolated after many binary decisions and has the longest path length on the isolation tree.

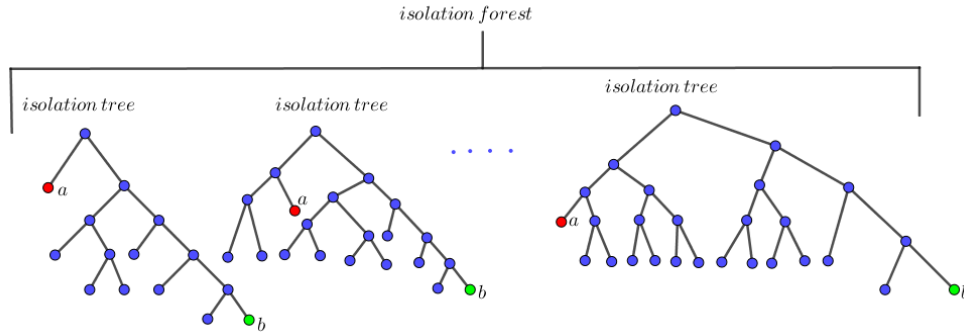


Figure 3.2: Demonstrates how the isolation tree was constructed and how the anomaly and normal data points were isolated.

The following definitions and formulae from Liu et al. (2008).

Definition 1: Isolation Tree. Let T be a node of an isolation tree. T is either an external node with no child or an internal node with one test and exactly two daughter nodes(T_l, T_r). X test consists of an attribute q and a split value p such that the test $q < p$ divides data points into T_l and T_r .

An isolation tree can be built from the dataset X where $X = \{x_1, x_2, x_3, \dots, x_n\}$ by following the same process that we discussed above in section(3.1.1). The isolation tree is a binary tree where each node in the tree has either two or zero child nodes. X subset of data that cannot be further partitioned is called an external node and the number of external nodes is equal to the number of data points in the dataset, which is n and the node that ends with two child nodes is called an internal node. The number of internal nodes in an isolation tree is equal to the number of partitioning operations performed minus one ($n-1$) because each internal node represents a partitioning operation and this is mean its required $n-1$ partitions in order to isolate all the data points in the dataset. Hence the total number of nodes in an isolation tree is equal to the sum of the external and internal nodes, which is $(2n-1)$. This makes the amount of memory needed constraining and only increases linearly when n is increased(i.e. during the construction of isolation trees, there is no need to store the actual data points or their feature values explicitly. Instead, the algorithm only needs to store the split feature, split value, and the depth of each tree node).

Definition 2: Path Length $h(x)$ of a point x is measured by the number of edges x traverses an isolation tree from the root node until the traversal is terminated at an external node.

An anomaly score for a data point is obtained by:

1. Calculating the path length $h(x)$ of the data point x in all isolation trees.
2. Calculating the average path length for the data point x across all isolation trees in which the data point is isolated.
3. Calculating anomaly score by averaging all the values obtained in the second step divided by the number of isolation trees in the forest. Obtained anomaly score ranges from 0 to 1 and a score

close to 1 means the data point is probably an anomaly and a score close to 0 means the data point is probably a normal point.

We can calculate the average path length of the isolation tree by first computing $c(n)$ where $c(n)$ is the average of path length $h(x)$ and n is the number of features in the dataset and since isolation tree has a similar structure of Binary Search Tree(BST), then the calculation of the average path length for an external node termination is similar to search in BST. Hence the average path length can be calculated from the formula:

$$c(n) = 2H_{(n-1)} - \left(\frac{2(n-1)}{n}\right). \quad (3.1.1)$$

Where $H(i)$ is the harmonic number and can be calculated from $\ln(i) + 0.5772156649$ (Euler-Mascheroni constant). $c(n)$ represents the average of the path length, it is used to normalize $h(x)$.

Where :

$$c(n) = \begin{cases} 2H_{(n-1)} - 2(n-1)/n & \text{if } n > 2, \\ 1 & \text{if } n = 2, \\ 0 & \text{otherwise .} \end{cases}$$

The anomaly score for a given point x can be calculated from the formula:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}. \quad (3.1.2)$$

Where $E(h(x))$ represents the expected average path length from a set of trees. From equation(3.1.2):

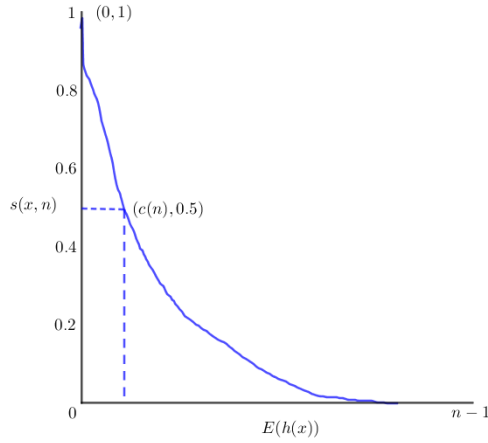
- when $E(h(x)) \rightarrow c(n)$, $s \rightarrow 0.5$;
- when $E(h(x)) \rightarrow 0$, $s \rightarrow 1$;
- when $E(h(x)) \rightarrow n-1$, $s \rightarrow 0$ (because the average path length to isolate an instance becomes closer to the length of the longest possible path in the forest, which is $n-1$. Consequently, the anomaly score which is calculated based on the average path length, approaches 0 for instances that are not outliers.)

s is monotonic for path length $h(x)$. Figure(3.3a) demonstrates the relationship between expected path length $E(h(x))$ and anomaly score s , taking into account the following conditions:

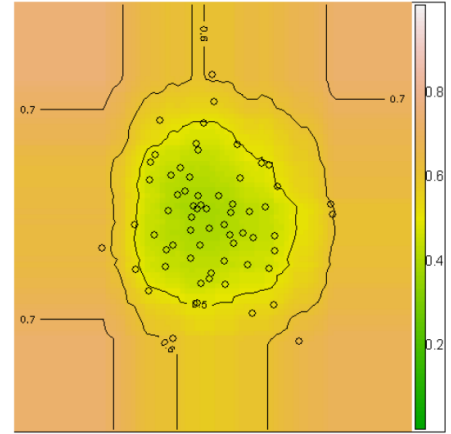
$$0 < s \leq 1 \text{ for } 0 < h(x) \leq n-1.$$

Using the anomaly score obtained from equation(3.1.2) the algorithm will be able to take one of these decisions:

- If the anomaly score of a data point is very close to 1, then the data point will be classified as anomalous.
- If the anomaly score of a data point is less than or equal to 0.5, then the data point will be classified as a normal point.
- If the anomaly score of all data points is the same as 0.5, then this indicates that there are no distinct anomalies in the dataset and the model will classify all the data points as normal points.
- If the expected path length $E(h(x))$ is the same as average path length $c(n)$, then the anomaly score $s = 0.5$, whatever the value of n is.



(a) relationship between $E(h(x))$ & $s(x, n)$ Liu et al. (2008).



(b) Anomaly score contour plot Liu et al. (2008).

Figure 3.3: Figure(3.3a) demonstrates the relationship between expected path length $E(h(x))$ and anomaly score $s(x, n)$. $c(n)$ represent the average path length in equation(3.1.1). Figure(3.3b) shows a contour plot for the anomaly score using Gaussian distribution for anomaly score $s = 0.5, 0.6, 0.7$ and the anomaly is a point that has anomaly score $s \geq 0.6$.

3.1.2 IForest Algorithms

There are two main steps in anomaly detection using IForest: Training and testing steps:

Training Step

In this step, the IForest algorithm builds an ensemble of isolation trees by recursively partitioning the given training dataset until each data point is isolated in its own leaf node or the tree height limit is reached. The tree height limit is given by the sub-sample size ψ : $l = \text{ceiling}(\log_2 \psi)$ which is approximately the average tree height. Algorithm 1 and 2 gives details about the training step.

Algorithm 1 : IForest (X, t, ψ)

Inputs: X - input data, t - number of trees, ψ - sub-sampling size

Output: a set of t isolation trees

- 1: Initialize Forest
 - 2: set height limit $l = \text{ceiling}(\log_2 \psi)$
 - 3: for $i = 1$ to t do
 - 4: $X' \leftarrow \text{sample}(X, \psi)$
 - 5: Forest $\leftarrow$ Forest $\cup$ Tree($X', 0, l$)
 - 6: end for
 - 7: return Forest
-

From the above algorithm, there are two input parameters to the IForest algorithm, sub-sampling size ψ and the number of trees to be built t .

Sub-sampling size controls the training data size and is practically found to be $\psi = 2^8 = 256$ by default. And this is a flexible value that the IForest algorithm performs well in detecting anomalies, but also it is not a fixed value for all datasets and can be changed according to the dataset.

The number of trees t controls the ensemble size and empirically the number of trees found to be $t = 100$ and this is because the path lengths usually converge well before $t = 100$, hence the number

of trees by default is $t = 100$. After the trees are constructed, then the model will be ready for the evaluation step.

Algorithm 2 : isolation tree (X, e, l)

Inputs: X - input data, e - current tree height, l - height limit

Output: an isolation tree

```

1: Initialize Forest,  $T = \phi$  (an empty tree)
2: if  $e \geq l$  or  $|X| \leq 1$  then
3:   return exNode{Size  $\leftarrow |X|$ }
4: else
5:   let set  $S$  to be a list of features in  $X$ 
6:   Randomly select a feature  $s \in S$ 
7:   Randomly select a split point  $p \in (\min(s) \text{ and } \max(s))$ 
8:   Define  $X_l$  and  $X_r$  contain the sample of  $X$ , where  $s$  is smaller and larger than  $p$ 
9:    $X_l \leftarrow \text{filter}(X, s < p)$ 
10:   $X_r \leftarrow \text{filter}(X, s \geq p)$ 
11:  return inNode {Left  $\leftarrow \text{isolation tree}(X_l, e + 1, l)$ ,
12:    Right  $\leftarrow \text{isolation tree}(X_r, e + 1, l)$ ,
13:    Split feature  $\leftarrow s$ ,
14:    Split Value  $\leftarrow p$ }
15: end if

```

Testing Step

In this step, the IForest algorithm uses the ensemble of trees to detect anomalies. Anomalies are identified as points that have high anomaly scores, and the anomaly scores can be obtained by calculating the average expected path length $E(h(x))$, and $h(x)$ can be calculated using the path length function(see algorithm 3). then anomaly score can be computed from equation(3.1.2).

Algorithm 3: Path Length (s, T, e)

Inputs : s - a feature, T - an isolation tree, e - current path length:

$s = 0$ in the first called

Output: path length of s

```

1: if  $T$  is an external node then
2:   return  $e + c(T)$  size { $c(\cdot)$  is defined in Equation (3.1.1)}
(This means if the current node is an external node then the path length
of the feature is the current path length plus the cost of the external node.)
3: end if
4:  $a \leftarrow T.\text{split feature}$ 
5: if  $s_a < T.\text{split Value}$  then
6:   return Path Length(  $s, T.\text{left}, e + 1$ )
7: else { $s_a \geq T.\text{split Value}$ }
8:   return Path Length( $s, T.\text{right}, e + 1$ )
9: end if

```

3.2 LOF Algorithm

LOF is a machine learning algorithm used for local outlier detection in a dataset which focuses on detecting data points that differ from their local neighbours more than detecting global outliers that

look different from the whole dataset. This algorithm works by firstly, computing the reachability distance for each point to its k -nearest neighbours (where k is determined by the user) and reachability distance is the maximum distance from a point to its k -nearest neighbours. Secondly, computing the local reachability density for each point by taking the inverse of the average reachability distance of its k -nearest neighbours. Finally, compute the local outlier factor score for each point as the ratio of its local reachability density to the average local reachability density of its k -nearest neighbours. And because this algorithm considers both the local density and the distances between points, this makes the algorithm detect even the points that are not isolated, that have a density lower than their neighbourhood and identify them as outlier points. After computing the LOF score, if a point has a LOF score greater than 1 it means that this point is more isolated than its neighbours and is classified as a local outlier. The following definitions explain LOF in detail and these definitions from [Breunig et al. \(2000\)](#).

3.2.1 Non-local Outlier Definitions

Definition 3: (Hawkins-Outlier)

An outlier is an observation that deviates so much from other observations as to arouse suspicion that it was generated by a different mechanism.

Points in the dataset will be denoted by p, q , and o (for illustration only). And d is the distance between points in the dataset, for instance, $d(p, h)$ indicates the distance between the two points p and h . And from the figure(1.1) we used (F , G , and E) to denote the cluster of points, and when we use $d(h, F)$ it indicates the minimum distance between h and point p in the cluster F , i.e. $d(h, F) = \min\{d(h, p) \mid p \in F\}$

Definition 4: (DB(pct, dmin)-Outlier)

An object p in dataset D is a DB(pct, dmin)-outlier if at least a percentage pct of the objects in D lies greater than the distance dmin from p , i.e., the cardinality of the set $\{q \in D \mid d(p, q) \leq dmin\}$ is less than or equal to $(100 - \text{pct})\%$ of the size of D .

The definition states that an object p in dataset D is considered a DB(pct, dmin)-outlier if the number of objects in D that are farther than dmin from p (i.e., the distance $d(p, q)$ between p and q is greater than dmin) is less than or equal to a certain proportion of the total number of objects in D . This proportion is calculated as $(100 - \text{pct})\%$ of the size of D . The above definition captures specific kinds of anomalies ("global outliers") because it takes the global view of the dataset. The global outliers are the data points that are far away from the dataset, but sometimes there are outliers within the cluster of points or very near to the cluster, and these are called "local outliers".

3.2.2 Local Outlier Definitions

Section (3.2.1) discussed a DB(pct, dmin)-outlier which is effective in specific circumstances, because it fails when there is a cluster with different densities. The main difference between the algorithm discussed in section(3.2.1) and LOF is that "being outlying is not a binary property".

Definition 5: (k-distance of an object p)

For any positive integer k , the k -distance of object p , denoted as $k\text{-distance}(p)$, is defined as the distance $d(p, o)$ between p and an object $o \in D$ such that:

- for at least k objects $o' \in D \setminus \{p\}$ it holds that $d(p, o') \leq d(p, o)$, and
- for at most $k-1$ objects $o' \in D \setminus \{p\}$ it holds that $d(p, o') < d(p, o)$.

Definition 6: (k-distance neighbourhood of an object p)

Given the k-distance of p, the k-distance neighbourhood of p contains every object whose distance from p is not greater than the k-distance, i.e. $N_{k-distance(p)}(p) = \{q \in D \setminus \{p\} \mid d(p, q) \leq k - distance(p)\}$. The k-nearest neighbour of p is represented by q points.

Definition 7: (reachability distance of an object p with respect to object o)

Let k be a natural number. The reachability distance of object p with respect to object o is defined as:

$$reach - dist_k(p, o) = \max\{k - distance(o), d(p, o)\}. \quad (3.2.1)$$

The definition states that the reachability distance of object p with respect to object o is determined by comparing two distances: the distance between object o and its k-th nearest neighbour, and the distance between object p and object o. The larger of the two distances is selected as the reachability distance.

Definition 8: (local reachability density of an object p)

The local reachability density of p is defined as:

$$lrd_k(p) = \frac{1}{\left[\frac{\sum_{o \in N_k(p)} reach-dist_k(p, o)}{|N_k(p)|} \right]}. \quad (3.2.2)$$

From the above equation, the local reachability distance is the inverse of the average reachability distance that we discussed in equation(3.2.1). Additionally, we can notice that if the reachability distance is zero the local reachability distance will be infinity and this happens when there are at least k duplicates of p in the dataset. To solve the duplicate issues the authors based their idea of the neighbourhood on a “k-distinct-distance” which is similar to the k-distance discussed in the definition(3.2.2), with an extra term that there be at least k points with various spatial coordinates.

Definition 9: ((local) outlier factor of an object p)

The local outlier factor of p is defined as:

$$LOF_k(p) = \frac{\sum_{o \in N_k(p)} \frac{lrd_k(o)}{lrd_k(p)}}{|N_k(p)|}. \quad (3.2.3)$$

The above formula returns a score for each data point. From this score, the point will be classified as normal or anomalous. The local outlier factor is almost 1 when the average local reachability density has the same value as the local reachability density. In this case, the point will be classified as a normal point. The local outlier factor is greater than 1 when the local reachability density is less than the average local reachability density.

The below algorithm shows how the local outlier factor works.

Algorithm4: LOF (k, m, D)

Input: k-number of nearest neighbours, m-number of outliers, D-outlier candidate dataset.

Output: top m outliers.

1 : for $j = 1$ to $\text{len}D$ do

2 : compute $k - dist(p)$

3 : compute $N_k(p)$

4 : end for

5 : calculate $reach - dist_k(p, q)$ and $lrd(p)$

6 : calculate $lof(p)$

7 : sort the lof values of all points in descending order

8 : return the m data objects with the large lof values, which are the outliers

The code for the IForest and LOF is uploaded to the repository and the link to the repository is [HERE](https://github.com/AbubakarMastour/AIMS-Project.git) or: (<https://github.com/AbubakarMastour/AIMS-Project.git>)

3.3 Evaluation Metrics

Evaluation metrics are quantitative measures used to assess the performance of a model and are employed in different fields, such as machine learning.

3.3.1 Matthew Correlation Coefficient (MCC)

A study conducted by [Matthews \(1975\)](#) proposed a statistical measure used for evaluating the performance of binary classification models. This tool considers true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) to provide a single measure of the model's predictive capability. The tool's values range from 1 to -1, where 1 indicates perfect prediction, 0 indicates random prediction, and -1 indicates the worst prediction, as shown in Figure 3.5. TP represents the number of correctly predicted positive cases (e.g., correctly predicting a signal), TN represents the number of correctly predicted negative cases (e.g., correctly predicting background), FP represents the number of falsely predicted positive cases (e.g., predicting a signal when it is background), and FN represents the number of falsely predicted negative cases (e.g., predicting background when it is a signal), as illustrated in Figure 3.6.

The score for MCC can be obtained from the following equation (3.3.1):

$$MCC = \frac{(TP \times TN - FP \times FN)}{\sqrt{(TP + FP) \times (TP + FN) \times (TN + FP) \times (TN + FN)}}. \quad (3.3.1)$$

The advantages of MCC compared to other metrics such as accuracy, precision, and recall is that MCC is not affected by class imbalance and takes into account both true and false predictions in a balanced way. It takes into account both false positives and false negatives which is crucial for anomaly detection. Also it does not rely on a specific decision threshold for classifying instances and only measures the overall agreement between predicted and actual labels regardless of the threshold used.

3.3.2 Confusion Matrix

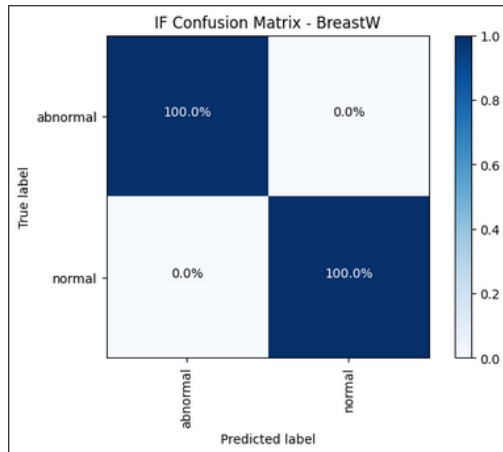
A confusion matrix is a statistical tool used to assess the performance of a model. It consists of four entries: true positive (TP), true negative (TN), false positive (FP), and false negative (FN). We discussed these four entries in section 3.3.1. Figure 3.4 illustrates the confusion matrix for the BreastW dataset [Wolpert and Macready \(1997\)](#) using the IForest and LOF algorithms.

3.3.3 Receiver Operating Characteristic (ROC) Curve

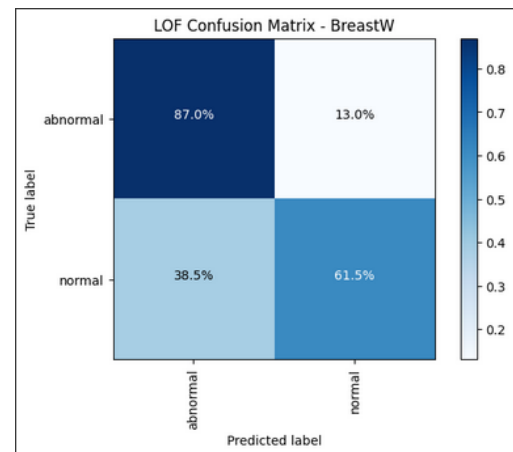
A study conducted by [Fawcett \(2006\)](#) defines the ROC curve as a graphical tool for evaluating and comparing the performance of binary classifiers. It plots two parameters: true positive rate (TPR) and false positive rate (FPR). This curve shows the relationship between FPR and TPR at different classification thresholds.

TPR: also known as recall, represents the proportion of positive attributes that are correctly classified as positive. It can be obtained using the equation (3.3.2) :

$$TPR = \frac{TP}{TP + FN}. \quad (3.3.2)$$



(a) confusion matrix for BreastW dataset using IForest algorithm.



(b) confusion matrix for BreastW dataset using LOF algorithm.

Figure 3.4: Demonstrate a plot of a confusion matrix for the BreastW dataset using IForest and LOF. The code for plotting the confusion matrix is from GitHub [elena sharova \(2023\)](#).

FPR: is the portion of negative attributes that are correctly classified as negatives, and can be obtained from this equation(3.3.3):

$$FPR = \frac{FP}{FP + FN}. \quad (3.3.3)$$

The dashed diagonal line from the bottom left corner to the top right corner in Figure 3.5 represents the

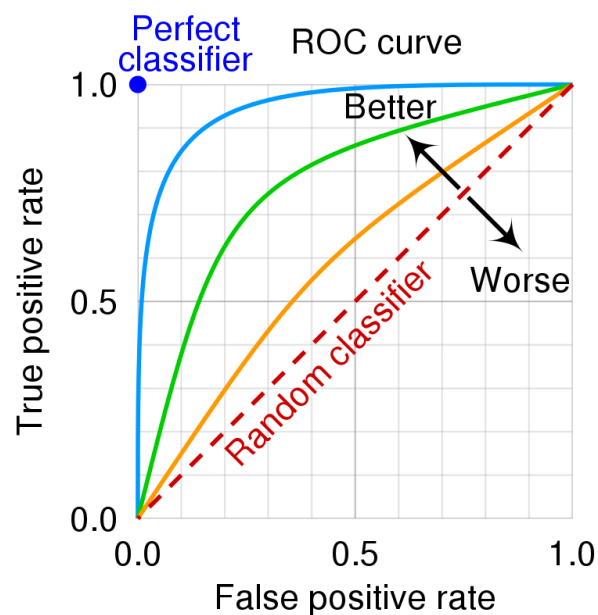


Figure 3.5: Demonstrates a graphical plot of false positive rate versus true positive rate as well as shows where the model performs well and where it performs badly in terms of AUC. Image courtesy [Wikipedia contributors \(2023\)](#)

classifier, the lines above the classifier mean that the true positive rate (TPR) is higher than the false

positive rate (FPR) at all classification thresholds, and if these lines are below the classifier it means that the TPR is lower than the FPR at all classification thresholds.

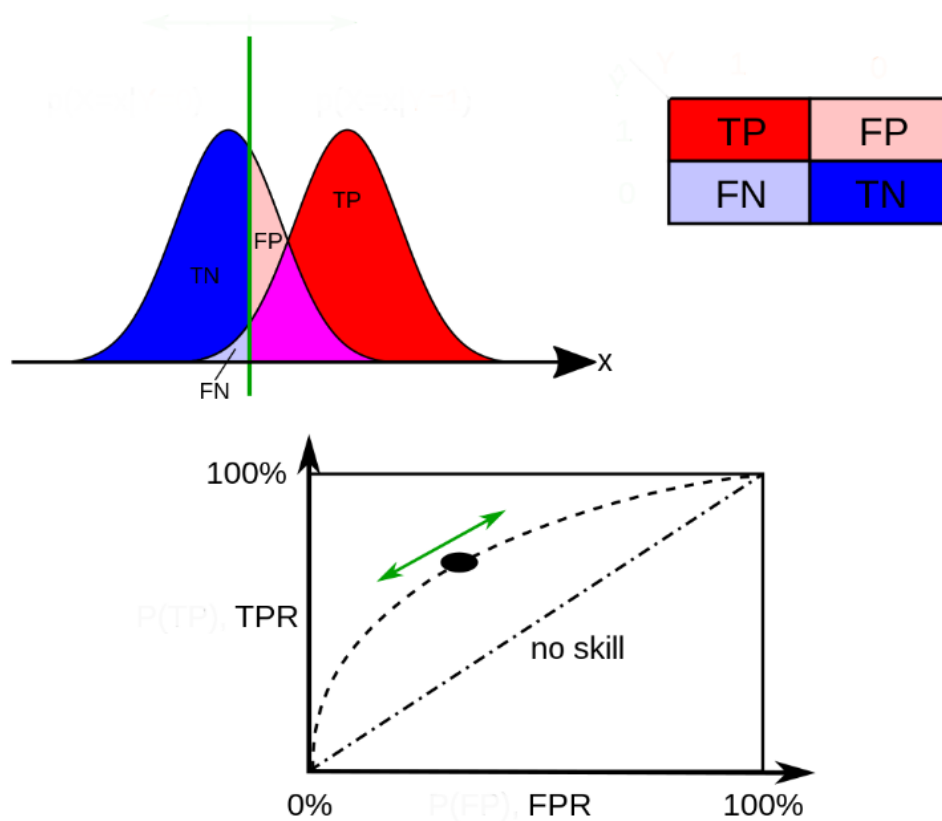


Figure 3.6: Demonstrates a graphical plot of true positive rate and false positive rate. Image courtesy [Wikipedia contributors \(2023\)](#)

4. Datasets

In this section, we will discuss the dataset that we are going to use to evaluate the performance of the IForest and LOF algorithms. These datasets are from Outlier Detection DataSet (ODDS), for further information, see [ODDS website](http://odds.cs.stonybrook.edu/) or (<http://odds.cs.stonybrook.edu/>). The description of the data is from the University of California Irvine(UCI) Machine Learning repository, for more information see [UCI repository](https://archive.ics.uci.edu/ml/index.php) or (<https://archive.ics.uci.edu/ml/index.php>).

Table 4.1 shows the characteristics and information of the datasets used to evaluate the two algorithms.

Name of the datasets	No. of Attributes of the dataset	Dimension of the dataset	Proportion of the anomalies in the dataset
Satellite	6435	36	32%
Shuttle	49097	9	7%
http(KDDCPU99)	567479	3	4%
ForestCover	286048	10	9%
Ionosphere	351	33	35%
BreastW	683	9	36%
Anthyroid	7200	6	7.42%

Table 4.1: shows the number of attributes and the dimensions of each dataset, as well as the percentage of anomalies for each dataset.

4.1 Satellite Dataset

The data is multispectral images of pixels in 3x3 neighbourhoods of different types of lands and this data is from [Srinivasan \(1993\)](#). The data was collected using Landsat satellite sensors for remote sensing and each image was taken using four different spectral bands, the first two bands are invisible bands represented in green and red and the two other bands are near-infrared and middle-infrared, and the main purpose of this dataset is to predict the land cover type from the spectral band(see figure 4.1). The dataset consists of 6,435 instances with 36 instances (four spectral bands times nine neighbouring pixels) extracted from the images each representing a pixel in the satellite image. For each pixel in the satellite image, the first 34 attributes are the reflectance values of the six spectral bands and the 35th attribute is a binary variable that shows whether the associated pixel is part of a training set or a test set. The 36th attribute is a class label which represents various forms of land cover and has a range of one of these six potential values (red soil, cotton crop, grey soil, damp grey soil, soil with vegetation stubble, and very damp grey soil).

4.2 Shuttle Dataset

This dataset is from [Catlett \(1988\)](#) and this dataset is used for classification tasks and contains 58000 instances with nine attributes and 1 nominal attribute. The nine attributes contain information about the launching of the space shuttle and the nominal attribute is a target class with a value of either 0 or 1. The main goal is to predict whether the launch was successful or not. If the launch succeeds then it is normal and takes the value 0 and if it failed then it is an anomaly and takes the value 1. This dataset is also used in different anomaly detection algorithms to detect abnormal behaviour or failure in the space shuttle sensors and detecting these anomalies helps for the safety of the space shuttle and its passengers.

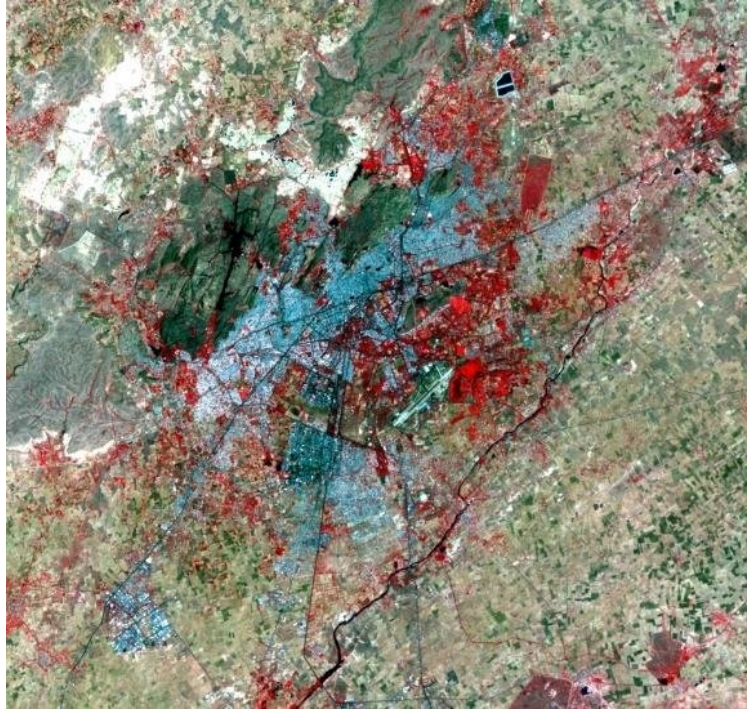


Figure 4.1: Illustrates image taken by Landsat satellite [Borana and Yadav \(2018\)](#).

4.3 Http(KDDCUP99) Dataset

This dataset is from [Stolfo and Chan \(1999\)](#) and is widely used in intrusion detection systems. The dataset contains a total of 4,000,000 network connection instances and 42 attributes. This dataset was created for the 1999 KDD CUP competition organized by the International Conference on Knowledge Discovery and Data Mining (KDD) and each connection record contains 41 attributes including the source and destination IP addresses as well as various indicators of network activity and behaviour.

The objective of the competition was to create models that could precisely classify each connection record as either attack or normal based on the values of the given attributes.

4.4 Forest Cover Dataset

This data is from [Blackard \(1998\)](#), and this dataset contains information about the forest cover type for various locations within Roosevelt National Forest in Colorado, United States of America such as elevation, slope, and soil type. Additionally, the dataset contains 581,012 samples each representing a 30×30 -meter cell with 54 features including both continuous and categorical variables and the target variable is the forest cover type which is a categorical variable with seven possible values these values represent forest cover types. The seven forest cover type classes are: lodgepole pine (*Pinus contorta*), spruce/fir (*Picea engelmannii* and *Abies lasiocarpa*), ponderosa pine (*Pinus ponderosa*), Douglas-fir (*Pseudotsuga menziesii*), aspen (*Populus tremuloides*), cottonwood/willow (*Populus angustifolia*, *Populus deltoides*, *Salix bebbiana*, *Salix amygdaloides*), and krummholz. The aim of the dataset is to classify the forest cover type based on the provided cartographic variables mentioned earlier. Class 2 was used to represent the normal class while class 4 was used to represent the anomaly class.

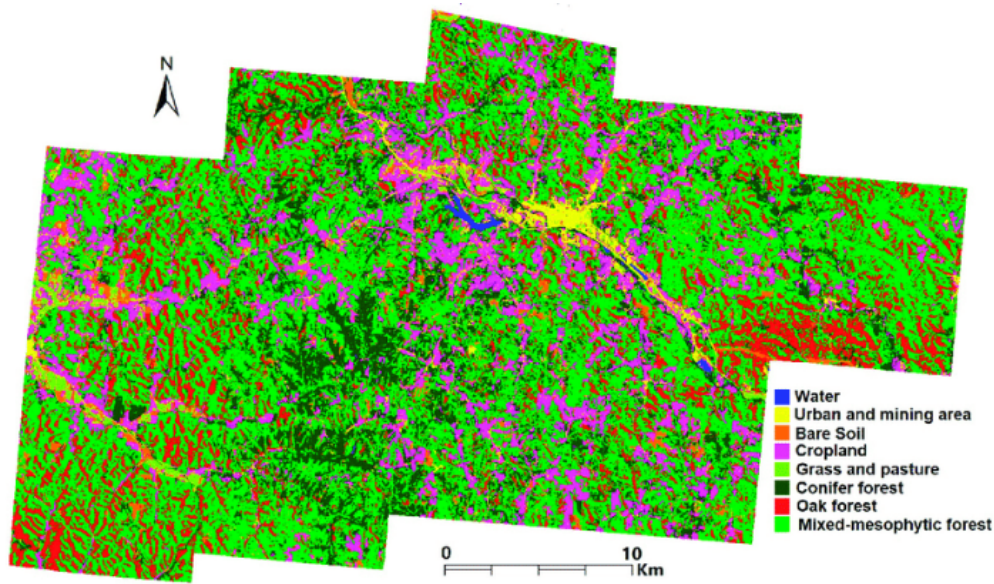


Figure 4.2: Forest cover type, from Landsat satellite. [Zhu and Liu \(2019\)](#).

4.5 Ionosphere Dataset

This dataset is from [Sigillito \(1989\)](#) and contains radar data collected by a radar system in Goose Bay, Labrador. This dataset consists of 351 instances and each instance contains 34 attributes including a binary class label. The first 33 attributes represent the radar data and the final attribute indicates whether the radar signal was reflected by the ionosphere or not the classes are labelled (“good”) representing a successful detection of the ionosphere and (“bad”) representing an unsuccessful detection of the ionosphere. The main goal of this dataset is to build a model that can precisely classify radar signals into either good or bad signals.

4.6 Breast Cancer Wisconsin Dataset

This dataset is from [Wolberg \(1992\)](#) and it consists of 699 instances of breast cancer tumours with 10 attributes describing the characteristics of the tumours. The attributes include the size of the tumour and the uniformity of cell size and shape. The first 9 attributes are real-valued features representing characteristics of the cell nuclei and the 10th attribute is the class label which is either 2 for benign or 4 for malignant. The main goal is to predict whether a tumour is benign or malignant based on its characteristics.

4.7 Annnthyroid Dataset

This dataset is from [Quinlan \(1987\)](#), and the dataset was collected at the Garvan Institute of Medical Research in Sydney, Australia. The dataset contains data from 3,772 patients, each represented by 21 attributes including the target variable. The target variable is a categorical variable indicating the patient’s diagnosis, which can be one of three classes: normal, hypothyroidism, or hyperthyroidism. The main goal of this dataset is to predict whether a patient has thyroid disease or not based on the given attributes.

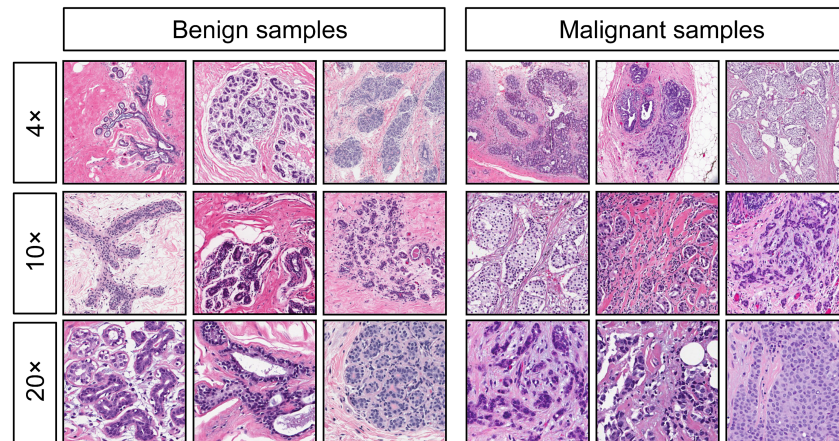


Figure 4.3: Shows benign samples and malignant samples. Image courtesy [Summary | Machine Learning for OpenCV](#).

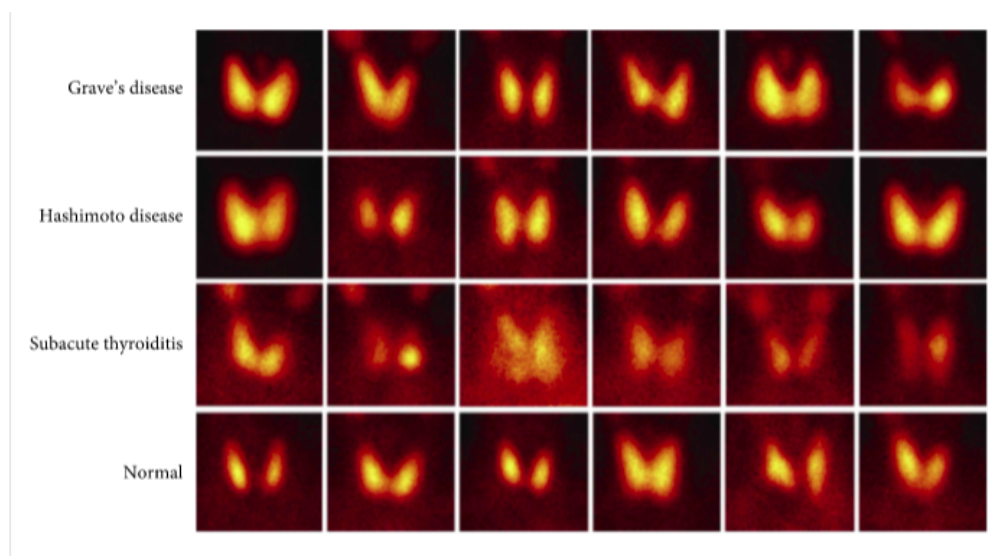
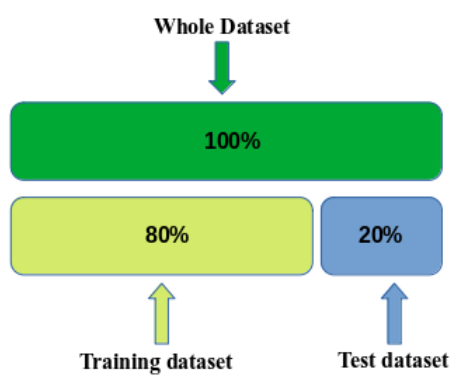


Figure 4.4: Thyroid Diagnosis image [Ma et al. \(2019\)](#)

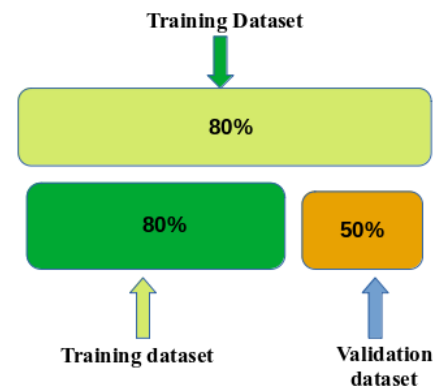
4.8 Splitting Datasets

At this stage, each dataset in each algorithm was divided into three parts: the training set, the testing set, and the validation set (see Figure 4.5). The training dataset is used to train each model by fitting it to the data and learning the underlying patterns. The validation dataset is used during the model development process to adjust the model's hyperparameters and evaluate its performance. The test dataset is used as the final evaluation step to assess the model's performance after all the model development and tuning steps have been completed.

Figure 4.5 is a flowchart describing how the dataset is split into train and test sets and then splits the training set into training and validation sets.



(a) divide a whole dataset into train and test sets.



(b) divide the training set into training and validation sets.

Figure 4.5: Figure(4.5a) demonstrates how the entire dataset has been divided into two subsets. Figure(4.5b) demonstrates how the training set on the other hand subdivided into training and validation sets.

4.9 Evaluation Stage

After splitting the dataset, train and validate the model. The evaluation of the performance of the model can be carried out using various metrics such as Confusion Matrix, Matthew Correlation Coefficient, and Receiver Operating Characteristic. However, in this case, we will only consider two metrics: Matthew Correlation Coefficient (MCC) and Receiver Operating Characteristic (ROC) curve.

5. Results and Discussions

In this section, two machine learning algorithms discussed in Chapter 3 were applied to seven different datasets. The performance of the two algorithms was evaluated using the Matthew Correlation Coefficient (MCC) and the Area Under the Curve (AUC), also known as the Receiver Operating Characteristic (ROC) curve.

5.1 Results

This section introduced the results that we have obtained from the two algorithms. The two algorithms are implemented in the “skit-learn” library, this library contains the two algorithms IForest and LOF. For IForest the model was trained using *model.fit()*, and the hyperparameter such as *contamination* was set in a for loop ranging from 0 to 50. For *max_samples* and *n_estimators* were manually optimized, and for prediction, we used *model.predict()*. For LOF, the model was trained using *lof.fit()*, and the hyperparameters such as *contamination* was set in a for loop ranging from 0 to 50, and *n_neighbors* was optimized manually, and for prediction, we used *lof.fit_predict()*.

Datasets	AUC		Execution Time(s)		MCC		Contamination %	
	IForest	LOF	IForst	LOF	IForst	LOF	IForst	LOF
BreastW	1.00	92	4.49	2.90	1.00	85	36	37
Shuttle	98	77	154.29	1740.62	96	85	7	9
Ionosphere	72	74	3.21	0.67	45	50	30	31
Satellite	69	73	23.79	207.15	48	55	16	20
Annthyroid	70	58	33.84	10.03	28	9	16	2
ForestCover	70	58	756.59	6996.17	14	42	27	2
Http(KDDCUP99)	1.00	98	1481.46	9524.83	60	33	1	3

Table 5.1: Demonstrate the result that we obtained using the two algorithms IForest and LOF.

5.1.1 IForest Result

As for IForest, we optimized the number of estimators for all seven datasets. The default value is 100 trees where the path length converges well before exceeding this number, and the number of trees is one of the hyperparameters in IForest. Another hyperparameter is *contamination*, which determines the proportion of anomalies to be detected in the dataset. Its value ranges from 0% to 50%, and to find the best performance of the model, we implemented a for loop that iterates over different contamination values. Within each loop, we calculated the model's performance and then we plot contamination values versus MCC to visualize the model's performance at different contamination levels (see Figure 5.2), and for the rest of the parameters and hyperparameters, we kept the default values.

Using BreastW and Http(KDDCUP99) datasets IForest achieved 100% AUC and in the BreastW, Http(KDDCUP99), Annthyroid, Shuttle, and Forest Cover datasets IForest outperforms LOF in terms of AUC. IForest outperforms LOF in terms of execution time in the datasets: Shuttle, Satellite, Http(KDDCUP99), and Forest Cover. In terms of MCC IForest outperforms LOF in BreastW, Shuttle, Annthyroid, and Http(KDDCUP99).

5.1.2 LOF Result

As for the LOF, it has two important hyperparameters: the number of nearest neighbours and the default value is 20 in the original algorithm. However, we used different numbers of neighbours for each dataset to obtain good performance. The second hyperparameter is contamination, we set up a for loop for the contamination, ranging from 0% to 50%, in order to find the contamination where the model performs well and for the rest of the parameters, we kept the default values. Figure 5.2 demonstrates the relationship between contamination and the Matthew Correlation Coefficient (MCC).

In terms of AUC LOF outperforms IForest in Ionosphere and Satellite datasets. In terms of execution time, LOF outperforms IForest in BreastW, Ionosphere, and Annthyroid datasets. In terms of MCC LOF outperforms IForest in Ionosphere, Satellite, and Forest Cover datasets.

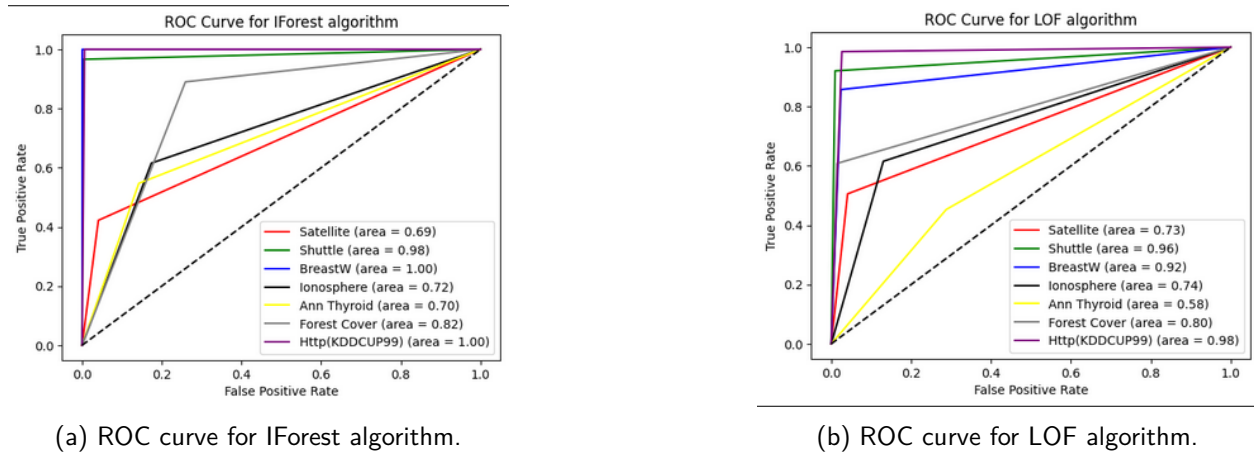


Figure 5.1: Figure(5.1a) demonstrates the ROC curve for the IForest algorithm and the values bottom right represent the AUC for each dataset. Figure(5.1b) demonstrates the ROC curve for the LOF algorithm and the values bottom right represent the AUC for each dataset.

After finding the contamination that achieved the best accuracy (MCC) we plot contamination versus MCC as shown in Figure (5.2), the value appearing in each figure represents the best MCC obtained from the respective algorithm, and the corresponding contamination is shown in Table 5.1. Then obtained AUC for each dataset by using the ROC curve, as shown in Figure 5.1. The ROC curve is obtained by plotting the TPR versus FPR, as discussed in Section 3.3.3.

These lines are not smooth because we used `predict()` function which applies binary threshold. To obtain smooth line we need to use a continuous output score. For IForest we can use `score_samples()` function, taking in our consideration that in this case, a lower value means more abnormal.

5.2 The Strengths and Weaknesses of the Two Algorithms

5.2.1 Advantages and Disadvantages of IForest

IForest has many properties that made this algorithm outperform other anomaly detection algorithms such as scalability, and this property enables the algorithm to perform well with large datasets and datasets with a large number of dimensions by utilizing random sampling to create sub-samples of the dataset. This algorithm also showed high efficiency in computational time, especially for large datasets, furthermore, is also robust to noise in the dataset because it isolates anomalies in separate partitions of the trees, moreover, the algorithm is flexible and can be applied to various data types, including

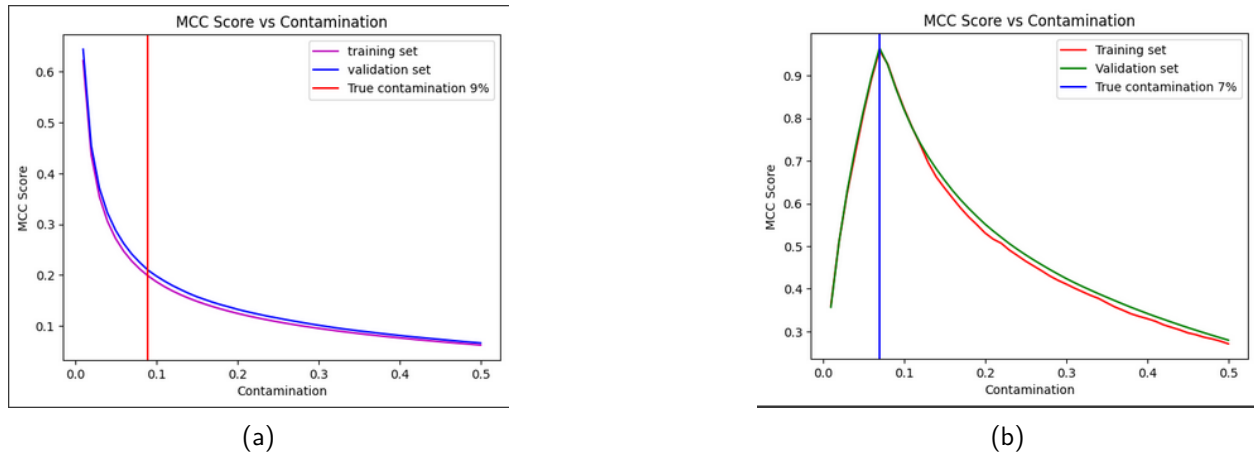


Figure 5.2: Figure(5.2a) demonstrates the relationship between training and validation sets for Http(KDDCUP99) dataset using IForest. Figure(5.2b) demonstrates the relationship between training and validation sets for the Shuttle dataset using LOF.

continuous and categorical data.

However, there are also weaknesses in this algorithm such as overfitting, the algorithm can suffer from overfitting especially when dealing with small datasets or the number of trees in the forest is too high or the dataset is highly clustered which can lead to result in low accuracy and poor performance, moreover, This algorithm also requires the setting of several hyperparameters, such as the number of trees in the forest, the sub-sampling size, contamination, and the maximum depth of the trees, these hyperparameters can have a significant impact on the performance of the algorithm and tuning them can consume a lot of time also the performance of the algorithm depends on the random selection of the partitioning feature and values which may lead to instability and variance in the results also this algorithm struggles with datasets that have dense clusters of instances, and in these cases, the algorithm may identify entire clusters as anomalies rather than individual instances. Furthermore, this algorithm suffers from swamping and masking, and swamping occurs when the normal points are too close to the anomalies, while masking happened when there is a big number of anomalies in the dataset.

5.2.2 Advantages and Disadvantages of LOF

LOF has many advantages over other anomaly detection algorithms such as identifying local outliers that are very difficult for other outlier detection algorithms to identify because it considers the local density of the data points, it can also handle large datasets because it only needs to compute distances between a data point and its k-nearest neighbour, as well as perform well in detecting outliers in high-dimensional datasets, furthermore, it also has the ability for identifying outliers in datasets with high levels of noise, and finally, it has also high flexibility to work with various types of datasets, including both numerical and categorical.

However, LOF has some limitations also; It is sensitive to hyperparameter selection, such as the number of neighbours, contamination, and the distance metric used to compute the local densities also, the performance of the algorithm decreased as the number of dimensions increases, also this algorithm may not perform well when the data has a complex distribution or when the outliers are distributed across multiple clusters. Additionally, it requires computing the distances between a point and its k-nearest neighbours, which can be computationally expensive for large datasets, also the performance of this algorithm may be sensitive to specific dataset characteristics such as the distribution of the data and

the degree of clustering present.

5.3 Discussion

When we compared the two algorithms based on the results obtained in Table 5.1, we found that IForest outperforms LOF in terms of execution time and this is because the IForest isolates anomalies instead of isolating normal instances as discussed in Section 3.1.1, on the other hand, LOF calculates the reachability distance for each data point to its k-nearest neighbours as discussed in Section 3.2, and this makes LOF takes more time to produce results compared to IForest. Furthermore, when we compared the two algorithms in terms of MCC we found that LOF outperforms IForest in some datasets, and in contrast, IForest outperforms LOF in other datasets. And when we compared the two algorithms in terms of the ROC curve we found that IForest outperforms LOF in terms of the area AUC in 5 datasets (BreastW, Http(KDDCUP99), Annthyroid, Shuttle, and Forest Cover datasets). Overall, the performance of the two algorithms is affected by the characteristics of the datasets and hyperparameter optimization, for that reason we found that in some datasets IForest outperforms LOF and in others LOF outperforms IForest. When we compared the results that we have obtained in table 5.1 with the results in the original Isolation Forest paper in table 2.1 we found that the IForest model achieved almost the same performance as in table 2.1 and this is because of the hyperparameter optimization, however, the LOF model achieved very good performance compared to the LOF performance in table 2.1. In fact, in some datasets, it even outperforms the LOF results in table 2.1.

6. Conclusion and future work

6.1 Conclusion

In this essay, we compare two machine learning algorithms: Isolation Forest and Local Outlier Factor discussed in sections 3.1 and 3.2 in terms of performance, and execution time. Additionally, we introduced the strengths and limitations of the two algorithms. The results show that the performance of each model depends on several parameters such as data types and hyperparameters. The two algorithms outperform each other in the seven datasets, with IForest performing better than LOF in some datasets. In terms of high-dimensional and large datasets, IForest demonstrates high performance than LOF and this is achieved in a very short time compared to LOF. This is because IForest isolate the anomalies rather than the normal points. The two algorithms are powerful in anomaly detection, especially when dealing with large amounts of data or a high-dimensional dataset. Therefore, it is advisable to try different anomaly detection algorithms on your datasets and compare their performance as there is no perfect algorithm for all types of datasets. The two algorithms suffer from some limitations, some of these limitations were solved by modifying the original version of these two algorithms, and we discussed some of these studies in Chapter 2.

6.2 Future Work

In future work, we will focus on active learning for anomaly detection, because this can help reduce the amount of labelled data needed for training anomaly detection algorithms while maintaining high accuracy in detecting anomalies.

Acknowledgements

I want to acknowledge AIMS and its funders for supporting this work, as well as my supervisor, Prof. Bruce Bassett from University of Cape Town for his support and guidance. My thanks extend to my tutor Emmanuel, Evelyn, and the academic director Dr. Simukai for their help and guidance, as well as my family and my friends for being always with me.

References

- Alghushairy, O., Alsini, R., Soule, T., and Ma, X. A review of local outlier factor algorithms for outlier detection in big data streams. *Big Data and Cognitive Computing*, 5(1):1, 2020.
- Alpaydin, E. *Introduction to machine learning*. MIT press, 2020.
- Blackard, J. Coverttype. UCI Machine Learning Repository, 1998. DOI: [10.24432/C50K5N](https://doi.org/10.24432/C50K5N).
- Borana, s. and Yadav, s. Ndvi-based vegetation changes and seasonal variation in semi arid region. 09 2018.
- Breunig, M. M., Kriegel, H.-P., Ng, R. T., and Sander, J. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 93–104, 2000.
- Catlett, J. Statlog (Shuttle). UCI Machine Learning Repository, 1988. DOI: [10.24432/C5WS31](https://doi.org/10.24432/C5WS31).
- Chandola, V., Banerjee, A., and Kumar, V. Anomaly detection: A survey. *ACM computing surveys (CSUR)*, 41(3):1–58, 2009.
- Cheng, Z., Zou, C., and Dong, J. Outlier detection using isolation forest and local outlier factor. In *Proceedings of the conference on research in adaptive and convergent systems*, pages 161–168, 2019.
- elena sharova. IsolationForest, June 2023. URL https://github.com/elena-sharova/IsolationForest/blob/master/IsolationForest_v0.1.ipynb. [Online; accessed 20. May. 2023].
- ELHadad, R., Tan, Y.-F., and Tan, W.-N. Comparison of enhanced isolation forest and enhanced local outlier factor in anomalous power consumption labelling. In *2023 IEEE 3rd International Conference in Power Engineering Applications (ICPEA)*, pages 243–247. IEEE, 2023.
- Fawcett, T. An introduction to roc analysis. *Pattern recognition letters*, 27(8):861–874, 2006.
- Hastie, T., Tibshirani, R., Friedman, J. H., and Friedman, J. H. *The elements of statistical learning: data mining, inference, and prediction*, volume 2. Springer, 2009.
- Heaton, J. Ian goodfellow, yoshua bengio, and aaron courville: Deep learning: The mit press, 2016, 800 pp, isbn: 0262035618. *Genetic Programming and Evolvable Machines*, 19(1-2):305–307, 2018.
- Jordan, M. I. and Mitchell, T. M. Machine learning: Trends, perspectives, and prospects. *Science*, 349(6245):255–260, 2015.
- Karczmarek, P., Kiersztyn, A., Pedrycz, W., and Al, E. K-means-based isolation forest. *Knowledge-based systems*, 195:105659, 2020.
- Laskar, M. T. R., Huang, J. X., Smetana, V., Stewart, C., Pouw, K., An, A., Chan, S., and Liu, L. Extending isolation forest for anomaly detection in big data via k-means. *ACM Transactions on Cyber-Physical Systems (TCPS)*, 5(4):1–26, 2021.
- Lesouple, J., Baudoin, C., Spigai, M., and Tournieret, J.-Y. Generalized isolation forest for anomaly detection. *Pattern Recognition Letters*, 149:109–119, 2021.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. Isolation forest. In *2008 eighth ieee international conference on data mining*, pages 413–422. IEEE, 2008.

- Liu, F. T., Ting, K. M., and Zhou, Z.-H. Isolation-based anomaly detection. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 6(1):1–39, 2012.
- Ma, L., Ma, C., Liu, Y., Wang, X., et al. Thyroid diagnosis from spect images using convolutional neural network with optimization. *Computational intelligence and neuroscience*, 2019, 2019.
- Marteau, P.-F., Soheily-Khah, S., and Béchet, N. Hybrid isolation forest-application to intrusion detection. *arXiv preprint arXiv:1705.03800*, 2017.
- Matthews, B. W. Comparison of the predicted and observed secondary structure of t4 phage lysozyme. *Biochimica et Biophysica Acta (BBA)-Protein Structure*, 405(2):442–451, 1975.
- Quinlan, R. Thyroid Disease. UCI Machine Learning Repository, 1987. DOI: [10.24432/C5D010](https://doi.org/10.24432/C5D010).
- Sigillito, W. S. H. L. . B. K., V. Ionosphere. UCI Machine Learning Repository, 1989. DOI: [10.24432/C5W01B](https://doi.org/10.24432/C5W01B).
- Srinivasan, A. Statlog (Landsat Satellite). UCI Machine Learning Repository, 1993. DOI: [10.24432/C55887](https://doi.org/10.24432/C55887).
- Stolfo, F. W. L. W. P. A., Salvatore and Chan, P. KDD Cup 1999 Data. UCI Machine Learning Repository, 1999. DOI: [10.24432/C51C7N](https://doi.org/10.24432/C51C7N).
- Summary | Machine Learning for OpenCV. Summary | machine learning for opencv. <https://subscription.packtpub.com/book/big-data-and-business-intelligence/9781783980284/5/ch05lvl1sec30/using-decision-trees-to-diagnose-breast-cancer>, May 2023. [Online; accessed 20. May 2023].
- Sutton, R. S. and Barto, A. G. *Reinforcement learning: An introduction*. MIT press, 2018.
- Tao, X., Peng, Y., Zhao, F., Zhao, P., and Wang, Y. A parallel algorithm for network traffic anomaly detection based on isolation forest. *International Journal of Distributed Sensor Networks*, 14(11): 1550147718814471, 2018.
- Vijayakumar, V., Divya, N. S., Sarojini, P., and Sonika, K. Isolation forest and local outlier factor for credit card fraud detection system. *International Journal of Engineering and Advanced Technology (IJEAT)*, 9:261–265, 2020.
- Wang, G. and Chen, Y. Robust feature matching using guided local outlier factor. *Pattern Recognition*, 117:107986, 2021.
- Wikipedia contributors. Receiver operating characteristic — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Receiver_operating_characteristic&oldid=1150954317, 2023. [Online; accessed 11-May-2023].
- Wolberg, W. Breast Cancer Wisconsin (Original). UCI Machine Learning Repository, 1992. DOI: [10.24432/C5HP4Z](https://doi.org/10.24432/C5HP4Z).
- Wolpert, D. H. and Macready, W. G. No free lunch theorems for optimization. *IEEE transactions on evolutionary computation*, 1(1):67–82, 1997.
- Xu, H., Zhang, L., Li, P., and Zhu, F. Outlier detection algorithm based on k-nearest neighbors-local outlier factor. *Journal of Algorithms & Computational Technology*, 16:17483026221078111, 2022.

- Zhu, X. and Liu, D. Investigating the impact of land parcelization on forest composition and structure in southeastern ohio using multi-source remotely sensed data. *Remote Sensing*, 11:2195, 09 2019. doi: 10.3390/rs11192195.